

INTERNSHIP TASK REPORT

Number Guessing Game

Week-01 | Task 2 | Khansa Malik

1. Objective

The objective of this task was to develop a Python-based number guessing game while applying software engineering practices that improve modularity, testability, and maintainability.

2. Design and Architecture

The application separates game logic from terminal interaction. The core logic is implemented in `game.py` as a `NumberGuessingGame` state machine with no `print()` or `input()` calls. It manages the secret number, attempts, guess history, and game state through a single `guess(value)` method that returns a typed `GuessResult`. The command-line interface is isolated in `cli.py` and is responsible only for user interaction and output formatting.

3. Key Implementation Features

Three difficulty levels are defined using a `Difficulty Enum`: Easy (1–50, 10 attempts), Medium (1–100, 7 attempts), and Hard (1–500, 10 attempts). The game provides immediate feedback for each guess, tracks attempts and guess history, reveals the answer after a loss, and supports replay. Invalid operations are handled through `InvalidGuessError` and `GameAlreadyOverError`.

4. Testability and Randomness

The game accepts an optional `random.Random` instance through dependency injection. This allows tests to use a seeded random number generator and obtain deterministic outcomes, making the test suite reliable and repeatable rather than dependent on unpredictable random values.

5. Testing

Automated testing was performed using `pytest`. A total of 12 unit tests passed successfully. The tests cover difficulty limits, correct guesses, too-low and too-high feedback, attempt tracking, exhaustion of attempts, post-game guesses, out-of-range inputs, and guess-history recording.

6. Outcome

The completed project is a functional and tested command-line game with a clear separation between application logic and presentation. Because the core game is independent of the terminal interface, the same `game.py` module could support a graphical or web-based front end without changes to its core logic.

7. Conclusion

This task demonstrated practical application of modular design, enumerations, exception handling, dependency injection, state management, and automated unit testing in Python. The resulting implementation is concise, maintainable, and structured for future extension.

Difficulty Configuration

Level	Range	Maximum Attempts
Easy	1–50	10
Medium	1–100	7
Hard	1–500	10